

Lab – ASP.NET - Uploading a Blob

To Begin with the Lab

Summary

In this lab, you learn how to upload a local file from a .NET application to an existing Azure Blob Storage Container. The application connects to the Azure Storage Account, creates a **BlobClient**, and uploads the file using the UploadAsync() method. The uploaded file (Blob) is stored securely inside the Blob Container and can be accessed whenever needed.

Understanding

Azure Blob Storage is used to store unstructured data such as documents, images, videos, backups, and application files. To upload a file, the application first connects to the Azure Storage Account using a **BlobContainerClient**. A **BlobClient** is then created for the file that will be uploaded. The UploadAsync() method copies the local file from the computer to the Blob Container. Setting the overwrite parameter to **true** ensures that if a blob with the same name already exists, it is replaced with the new file instead of generating an error.

Example

Suppose you have a Python file named **01.py** stored on your computer.

Local File:

C:\Code\01.py

Blob Container:

images

Blob Name:

01.py

After running the application, the file is uploaded as shown below:

Local Computer

|



C:\Code\01.py

|



BlobContainerClient

|



Blob Container (images)

|



01.py (Blob)

The file **01.py** is now stored inside the **images** Blob Container in Azure Storage.

Lab Steps

Step 1: Open the .NET Project

- Open your .NET project in Visual Studio or Visual Studio Code.

- Ensure the project is already connected to the Azure Storage Account.
- Verify that the Blob Container has already been created.

Step 2: Create the Upload Method

- Create a method to upload a local file to the Blob Container.
- static async Task UploadBlob(BlobContainerClient containerClient)

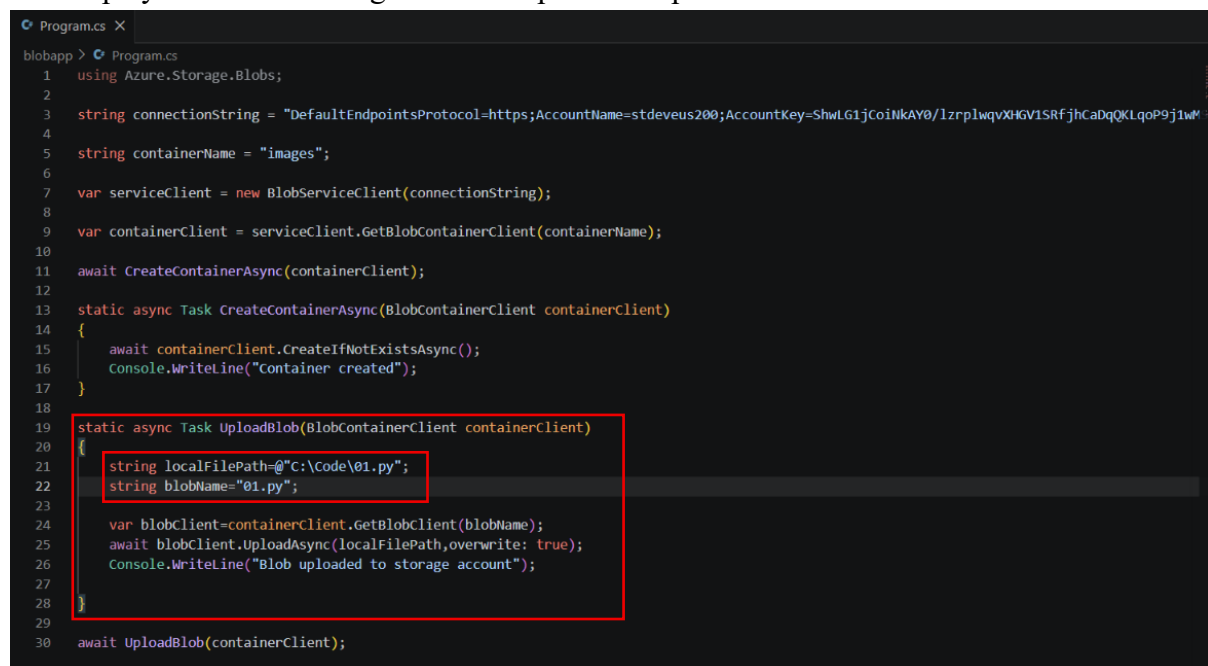
```
{
    string localFilePath = @" ";
    string blobName = " ";

    var blobClient = containerClient.GetBlobClient(blobName);

    await blobClient.UploadAsync(localFilePath, overwrite: true);

    Console.WriteLine("Blob uploaded to storage account");
}
```

- Specify the local file path.
- Specify the blob name.
- Create a BlobClient.
- Upload the file to Azure Blob Storage.
- Display a success message after the upload completes.



```
Program.cs X
blobapp > Program.cs
1  using Azure.Storage.Blobs;
2
3  string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdevous200;AccountKey=ShwL61jCoiMkAY0/1zrp1wqvXHGv1SRfjhCaDqQKLqoP9j1wM";
4
5  string containerName = "images";
6
7  var serviceClient = new BlobServiceClient(connectionString);
8
9  var containerClient = serviceClient.GetBlobContainerClient(containerName);
10
11 await CreateContainerAsync(containerClient);
12
13 static async Task CreateContainerAsync(BlobContainerClient containerClient)
14 {
15     await containerClient.CreateIfNotExistsAsync();
16     Console.WriteLine("Container created");
17 }
18
19 static async Task UploadBlob(BlobContainerClient containerClient)
20 {
21     string localFilePath = @"C:\Code\01.py";
22     string blobName = "01.py";
23
24     var blobClient = containerClient.GetBlobClient(blobName);
25     await blobClient.UploadAsync(localFilePath, overwrite: true);
26     Console.WriteLine("Blob uploaded to storage account");
27 }
28
29 await UploadBlob(containerClient);
30
```

Step 3: Call the Upload Method

- Call the upload method after creating the BlobContainerClient.

```
await UploadBlob(containerClient);
```

- Executes the upload process.
- Transfers the local file to Azure Blob Storage.

```

Program.cs X
blobapp > Program.cs
1 using Azure.Storage.Blobs;
2
3 string connectionString = "DefaultEndpointsProtocol=https;AccountName=stdevs200;AccountKey=ShwL61jCoInkAY0/1zrp1wqvXHGV1SRfjhCaDqQKLqoP9j1wW";
4
5 string containerName = "images";
6
7 var serviceClient = new BlobServiceClient(connectionString);
8
9 var containerClient = serviceClient.GetBlobContainerClient(containerName);
10
11 await CreateContainerAsync(containerClient);
12
13 static async Task CreateContainerAsync(BlobContainerClient containerClient)
14 {
15     await containerClient.CreateIfNotExistsAsync();
16     Console.WriteLine("Container created");
17 }
18
19 static async Task UploadBlob(BlobContainerClient containerClient)
20 {
21     string localFilePath=@"C:\Code\01.py";
22     string blobName="01.py";
23
24     var blobClient=containerClient.GetBlobClient(blobName);
25     await blobClient.UploadAsync(localFilePath,overwrite: true);
26     Console.WriteLine("Blob uploaded to storage account");
27 }
28
29
30 await UploadBlob(containerClient);

```

Step 4: Build the Project

Run the following command:

```
dotnet build
```

- Compiles the project.
- Checks for build errors.

Expected Output:

Build succeeded.

```

powershell blobapp X
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container/blobapp> dotnet build
Restore complete (0.5s)
blobapp net10.0 succeeded (3.9s) -> bin\Debug\net10.0/blobapp.dll
Build succeeded in 5.0s
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container/blobapp>

```

Step 5: Run the Application

Run the following command:

```
dotnet run
```

- Starts the .NET application.
- Uploads the local file to the Blob Container.

Expected Output:

Blob uploaded to storage account

```

powershell blobapp X
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container/blobapp> dotnet build
Restore complete (0.5s)
blobapp net10.0 succeeded (3.9s) -> bin\Debug\net10.0/blobapp.dll
Build succeeded in 5.0s
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container/blobapp> dotnet run
Container created
Blob uploaded to storage account
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container/blobapp>

```

Step 6: Verify the Uploaded Blob

- Open the Azure Portal.

- Navigate to your **Storage Account**.
- Select **Data Storage** → **Containers**.
- Open the Blob Container (for example, **images**).

Home > stdeveus200_1785129163420 | Overview > stdeveus200

stdeveus200 | Containers ☆ ...

Storage account

Search

+ Add container Upload Refresh Delete Change access level Restore containers Edit columns

Search containers by prefix Only show active containers

Showing all 2 items

☐	Name	Last modified	Anonymous access level	Lease state	Storage connector
☐	blobs	7/27/2026, 10:43:15 AM	Private	Available	-
☐	images	7/27/2026, 11:10:44 AM	Private	Available	-

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Storage Mover

Partner solutions

Resource visualizer

Data storage

Containers ☆

- Verify that **01.py** appears in the list of blobs.

Home > stdeveus200_1785129163420 | Overview > stdeveus200 | Containers

images ...

Container

Search

+ Add Directory Upload Change access level Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Shared access tokens

Access policy

Properties

Metadata

images

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive) Only show active blobs

Showing all 1 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	01.py	7/27/2026, 12:23:17 PM	Hot (Inferred)	Block blob	313 B	Available